

Code Top 100 — 手写算法完整导航

🎯 面试星级：★★★★★ | 建议用时：持续刷题

Code Top 100 算法题解，覆盖哈希表、双指针、链表、二叉树、动态规划、回溯、图等高频面试算法

文档更新时间：2026年5月17日

总题数：100 个条目（含重复索引题）

覆盖范围：数据结构 + 高频算法 + 设计题 + JavaScript ES6+ 手写实现

适用场景：前端面试 / 算法复盘 / 手写题训练

本文特色：每题都有「一句话思路」+「记忆口诀」+「速记模板」+「通俗解释」，帮你 3 分钟理解、5 分钟背熟、面试直接写。

🎯 快速导航

按难度等级

- 🟢 **简单：** 27 题，适合作为每日热身和基础手写题。
- 🟡 **中等：** 63 题，面试主战场，重点训练思路迁移。
- 🔴 **困难：** 10 题，用来突破边界处理、复杂状态和综合设计。

按刷题阶段

- 快速入门**（1-2 天）→ 哈希表、双指针、链表基础、有效括号。
- 核心强化**（1-2 周）→ 树、动态规划、二分、滑动窗口、TopK。
- 面试冲刺**（3-4 周）→ LRU、K 个一组翻转、最大路径和、编辑距离、接雨水。

按题型能力

- 状态记录：** 哈希表、前缀和、Map/Set。
- 边界控制：** 双指针、滑动窗口、二分查找。
- 结构操作：** 链表、二叉树、栈、队列、堆。
- 状态转移：** 动态规划、贪心、回溯。
- 工程化手写：** LRU、Rand10、字符串解析、原地矩阵操作。

📖 完整题目清单

分类索引

模块	题号范围	数量	难度分布	核心能力	优先掌握
1 哈希表	1-3	3	🟢 1 🟡 2 🔴 0	Map/Set、前缀和、去重查找	1、2、3

模块	题号范围	数量	难度分布	核心能力	优先掌握
2 双指针与滑动窗口	4-11	8	2 4 2	左右指针、窗口收缩、原地交换	4、5、7、8、11
3 链表操作	12-23	12	5 6 1	dummy 节点、快慢指针、局部反转	12、13、14、17、20、21
4 二叉树遍历与属性	24-41	18	8 9 1	DFS/BFS、BST、LCA、路径问题	24、26、29、33、41
5 动态规划	42-53	12	1 9 2	状态定义、转移方程、滚动优化	42、43、44、47、48、52
6 字符串处理	54-65	12	3 9 0	大数运算、括号、解析、回溯	54、55、56、59、63、65
7 二分查找	66-71	6	2 3 1	有序性、边界定位、答案二分	66、69、70、71
8 栈与队列	72-75	4	2 1 1	栈模拟、最小栈、单调队列	73、74、75
9 排序与 TopK	76-80	5	0 4 1	快排、堆、归并、区间排序	76、77、78、79
10 回溯与递归	81-85	5	0 5 0	选择列表、撤销选择、剪枝	81、82、83、85
1 1 DFS/BFS/图	86-90	5	0 5 0	网格 DFS、BFS、拓扑排序	86、87、88
1 2 设计题	91-93	3	1 2 0	API 不变量、缓存、拒绝采样	91、93
1 3 其他经典	94-100	7	2 4 1	贪心、投票、原地矩阵、排列	94、96、97、98、100

重复索引说明

- **72 / 55**: 有效的括号。
- **84 / 63**: 括号生成。
- **89 / 27**: 二叉树的右视图。
- **90 / 24**: 二叉树的层序遍历。
- **92 / 73**: 用栈实现队列。

学习路径建议

第1周：线性结构与指针基础

目标：掌握数组、哈希、双指针、链表的常见套路。

1. **哈希表：**两数之和、和为 K 的子数组、最长连续序列
2. **双指针/滑动窗口：**无重复字符最长子串、三数之和、最小覆盖子串
3. **链表：**反转链表、合并有序链表、环形链表、K 个一组翻转

第1周小测：手写 1 道哈希、1 道滑窗、2 道链表题。

第2周：树、栈、队列与搜索

目标：熟悉递归、层序遍历和辅助结构的使用。

1. **二叉树：**层序遍历、最近公共祖先、验证 BST、最大路径和
2. **栈与队列：**有效括号、用栈实现队列、最小栈、滑动窗口最大值
3. **DFS/BFS/图：**岛屿数量、岛屿最大面积、课程表

第2周小测：树题 3 道 + 栈队列题 2 道 + 图题 1 道。

第3-4周：动态规划与搜索边界

目标：把"状态是什么、怎么转移、如何初始化"练成肌肉记忆。

1. **DP 入门：**爬楼梯、最大子数组和、打家劫舍、最小路径和
2. **DP 进阶：**LIS、LCS、编辑距离、最长有效括号
3. **二分查找：**二分查找、旋转数组、首尾位置、中位数

第3-4周小测：每天 1 道 DP，隔天补 1 道二分边界题。

第5-6周：综合题与面试冲刺

目标：训练在陌生题面里快速定位题型、选数据结构、写稳定代码。

1. **排序与 TopK：**快速排序、第 K 大、合并 K 个链表、合并区间
2. **回溯与字符串：**全排列、组合总和、子集、字符串解码、计算器
3. **设计与其他：**LRU、Rand10、缺失第一个正数、旋转图像、下一个排列

第5-6周小测：限时 45 分钟完成 1 道中等题，并补复杂度分析。

高频面试题 TOP 25

基础必刷

1. 两数之和 
2. 无重复字符最长子串 
3. 三数之和 
2. 反转链表 
5. 合并两个有序链表 

数据结构核心

6. 删除链表倒数第 N 个 ☒ 7. 环形链表 II ☒ 8. 二叉树层序遍历 ☒
7. 最近公共祖先 ☒ 10. 验证 BST ☒

算法高频

11. 最大子数组和 ☒ 12. 最长上升子序列 ☒ 13. 编辑距离 ☒
12. 搜索旋转排序数组 ☒ 15. 第 K 个最大元素 ☒

难点突破

16. 接雨水 ☒ 17. 最小覆盖子串 ☒ 18. K 个一组翻转 ☒
17. 二叉树最大路径和 ☒ 20. 两正序数组中位数 ☒

手写设计

21. LRU 缓存 ☒ 22. 用栈实现队列 ☒ 23. 最小栈 ☒
22. 基本计算器 II ☒ 25. 缺失第一个正数 ☒

学习建议

每日安排

- **早**：复盘前一天错题和核心模板（15分钟）
- **中**：新刷 1 道题，先写思路再写代码（45-60分钟）
- **晚**：重写代码 + 补复杂度 + 记录易错点（30分钟）

学习方法

1. **先分类**：看到题目先判断是哈希、双指针、DP、树、图还是设计
2. **写不变量**：明确循环或递归过程中始终成立的条件
3. **补边界**：空数组、单节点、重复值、负数、溢出、头尾删除都要过一遍
4. **重写一遍**：当天会写不算稳，隔天能无提示写出来才算掌握

效率提升

- ☒ 先做 TOP 25，再补完整 100 题
 - ☒ 中等题优先，因为最贴近面试主战场
 - ☒ 困难重点学套路，不必死记每一行
 - ☒ 每题至少说清楚"为什么是这个数据结构"
-

一、哈希表

💡 **万能模板**：哈希表 = "边遍历边记录"。用 Map/Set 存已经见过的信息，每次查"之前有没有能配对的"。

💡 **记忆口诀**：先查后存，避免自配；前缀和题，预置零一

⚠️ **常见陷阱**：

- 前缀和必须预置 `Map.set(0, 1)`，否则从下标 0 开始的子数组会漏掉
- 先查再存，防止当前元素和自己配对
- 连续序列题：只从序列起点开始计数 (`!set.has(num-1)`)，保证 $O(n)$

1. 两数之和 ●

频度：301 | 难度：简单 | **LeetCode 1**

链接：[LeetCode 1](#)

题意：给定整数数组和目标值，找到两个不同下标使元素之和等于目标值。

示例：`nums = [2,7,11,15]`, `target = 9` → `[0,1]`

💡 **一句话思路**：遍历数组，用 Map 记录每个数的下标，每次查 `target - 当前数` 是否在 Map 中。

💡 **记忆口诀**：边走边查，找到就回

📖 **通俗解释**：就像在人群中找两个人身高加起来等于指定值。从左往右走，每遇到一个人先看"之前有没有人能配对"，有就找到；没有就把这个人的身高和位置记下继续走。

⚡ **速记模板**：

```
function twoSum(nums, target) {  
  const map = new Map();  
  for (let i = 0; i < nums.length; i++) {  
    const need = target - nums[i];  
    if (map.has(need)) return [map.get(need), i];  
    map.set(nums[i], i);  
  }  
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

2. 和为 K 的子数组 ●

频度：78 | 难度：中等 | **LeetCode 560**

链接：[LeetCode 560](#)

题意：统计元素和等于 `k` 的连续子数组数量。

示例：`nums = [1,1,1]`, `k = 2` → 2

💡 **一句话思路**：前缀和 + 哈希表，查 `sum - k` 出现过几次。

💡 **记忆口诀**：前缀和相减等于 K，查差值出现了几次

📄 **通俗解释**：每天记录累计花了多少钱。想知道"哪几天之间花了正好 K 块"，就看"今天累计 - K"以前出现过几次。
preMap.set(0, 1) 必须预置！

⚡ **速记模板**：

```
function subarraySum(nums, k) {  
  const preMap = new Map();  
  preMap.set(0, 1);  
  let sum = 0, count = 0;  
  for (const num of nums) {  
    sum += num;  
    count += preMap.get(sum - k) || 0;  
    preMap.set(sum, (preMap.get(sum) || 0) + 1);  
  }  
  return count;  
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

3. 最长连续序列 ●

频度：91 | 难度：中等 | LeetCode 128

链接：[LeetCode 128](#)

题意：未排序数组中数字值连续的最长序列长度。

示例：[100,4,200,1,3,2] → 4 (序列 1,2,3,4)

💡 **一句话思路**：Set 去重，只从序列起点 (num-1 不在 Set 中) 开始往后数。

💡 **记忆口诀**：只从起点数，避免重复算

📄 **通俗解释**：只从"没有前一张卡片"的数字开始数。每个数字最多被访问 2 次，保证 $O(n)$ 。

⚡ **速记模板**：

```
function longestConsecutive(nums) {  
  const set = new Set(nums);  
  let maxLen = 0;  
  for (const num of set) {  
    if (!set.has(num - 1)) {  
      let cur = num, count = 1;  
      while (set.has(cur + 1)) { cur++; count++; }  
      maxLen = Math.max(maxLen, count);  
    }  
  }  
  return maxLen;  
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

二、双指针与滑动窗口

💡 **万能模板**：双指针 = "两个游标，同向或相向移动，不回退"。滑动窗口 = "右指针扩张，左指针收缩"。

💡 **记忆口诀**：右扩左缩，同向不回；排序去重，跳过相同

⚠️ **常见陷阱**：

- 滑动窗口：左指针收缩条件要写对（满足条件才缩）
- 三数之和：必须先排序，去重用 `nums[i] === nums[i-1]`
- 接雨水：木桶效应，水量由较矮的一侧决定

4. 无重复字符的最长子串 ●

频度：1140 | 难度：中等 | LeetCode 3

题意：不含重复字符的最长连续子串长度。

示例："abcabcbb" → 3 ("abc")

💡 **一句话思路**：滑动窗口 + Map 记录字符最后位置，遇到重复左指针跳到重复字符下一个。

💡 **记忆口诀**：右走记录位置，重复就跳左

📖 **通俗解释**：每读一个新字母，看它上次出现在哪。如果上次位置在当前窗口内，左边界跳到那的下一个。
`map.get(s[r]) >= l` 判断很重要！

⚡ **速记模板**：

```
function lengthOfLongestSubstring(s) {
  const map = new Map();
  let maxLen = 0, l = 0;
  for (let r = 0; r < s.length; r++) {
    if (map.has(s[r]) && map.get(s[r]) >= l) l = map.get(s[r]) + 1;
    map.set(s[r], r);
    maxLen = Math.max(maxLen, r - l + 1);
  }
  return maxLen;
}
```

复杂度：时间 $O(n)$ ，空间 $O(\Sigma)$

5. 三数之和 ●

频度：477 | 难度：中等 | LeetCode 15

题意：找出所有不重复三元组使三数之和为 0。

💡 **一句话思路**：排序后固定第一个数，剩下两个用左右指针从两端向中间靠拢。

💡 **记忆口诀**：排序定一，左右靠拢；和为零记录，去重跳过

📖 **通俗解释**：排好队后挑第一个人（固定 i ），剩下两人从最左和最右往中间走。和太小左移，和太大右移，等于 0 记录并跳过重复。

⚡ 速记模板:

```
function threeSum(nums) {
  const result = [];
  nums.sort((a, b) => a - b);
  for (let i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] === nums[i-1]) continue;
    let l = i + 1, r = nums.length - 1;
    while (l < r) {
      const sum = nums[i] + nums[l] + nums[r];
      if (sum === 0) {
        result.push([nums[i], nums[l], nums[r]]);
        while (l < r && nums[l] === nums[l+1]) l++;
        while (l < r && nums[r] === nums[r-1]) r--;
        l++; r--;
      } else if (sum < 0) l++;
      else r--;
    }
  }
  return result;
}
```

复杂度: 时间 $O(n^2)$, 空间 $O(\log n)$

6. 合并两个有序数组 ●

频度: 294 | 难度: 简单 | LeetCode 88

题意: 将两个非递减数组合并到第一个数组中。

💡 **一句话思路:** 从后往前比较, 把大的放到 nums1 末尾, 避免覆盖。

💬 **记忆口诀:** 从后往前, 大的先放

📄 **通俗解释:** nums1 后面有空位。从后往前放不会覆盖任何有效数据。

⚡ 速记模板:

```
function merge(nums1, m, nums2, n) {
  let p1 = m-1, p2 = n-1, p = m+n-1;
  while (p1 >= 0 && p2 >= 0) {
    nums1[p--] = nums1[p1] > nums2[p2] ? nums1[p1--] : nums2[p2--];
  }
  while (p2 >= 0) nums1[p--] = nums2[p2--];
}
```

复杂度: 时间 $O(m+n)$, 空间 $O(1)$

7. 接雨水 ●

频度：196 | 难度：困难 | LeetCode 42

题意：柱状图高度数组，计算能接住多少水。

💡 **一句话思路**：双指针从两端走，水量由较矮侧决定（木桶效应）。

🧠 **记忆口诀**：哪边矮算哪边，矮侧水量 = 矮侧最大 - 当前高度

📖 **通俗解释**：下雨时水能存多少取决于两边围墙哪个更矮。左边矮就处理左边，右边矮就处理右边。

⚡ **速记模板**：

```
function trap(height) {
  let l = 0, r = height.length - 1, leftMax = 0, rightMax = 0, ans = 0;
  while (l < r) {
    if (height[l] < height[r]) {
      height[l] >= leftMax ? leftMax = height[l] : ans += leftMax - height[l];
      l++;
    } else {
      height[r] >= rightMax ? rightMax = height[r] : ans += rightMax - height[r];
      r--;
    }
  }
  return ans;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

8. 最小覆盖子串 ●

频度：130 | 难度：困难 | LeetCode 76

题意：s 中包含 t 全部字符的最短子串。

💡 **一句话思路**：滑动窗口 + 计数器，右扩凑齐，左缩找短。

🧠 **记忆口诀**：右扩凑齐，左缩找短；matched 等于 need.size 就收缩

📖 **通俗解释**：像超市找清单商品。右指针往购物车放直到凑齐，左指针往外拿看能不能更小。

⚡ **速记模板**：

```
function minWindow(s, t) {
  const need = new Map();
  for (const c of t) need.set(c, (need.get(c) || 0) + 1);
  let l = 0, r = 0, matched = 0, minLen = Infinity, start = 0;
  const window = new Map();
  while (r < s.length) {
    const c = s[r];
    window.set(c, (window.get(c) || 0) + 1);
    if (need.has(c) && window.get(c) === need.get(c)) matched++;
    r++;
  }
```

```

while (matched === need.size) {
  if (r - l < minLen) { minLen = r - l; start = l; }
  const leftChar = s[l];
  window.set(leftChar, window.get(leftChar) - 1);
  if (need.has(leftChar) && window.get(leftChar) < need.get(leftChar)) matched--;
  l++;
}
}
return minLen === Infinity ? "" : s.substr(start, minLen);
}

```

复杂度：时间 $O(n)$ ，空间 $O(|t|)$

9. 长度最小的子数组 ●

频度：72 | 难度：中等 | LeetCode 209

题意：和至少为 target 的最短连续子数组长度。

💡 一句话思路：右指针加数， $\text{sum} \geq \text{target}$ 时左指针减数并更新最短长度。

💬 记忆口诀：右加左减，够就缩

⚡ 速记模板：

```

function minSubArrayLen(target, nums) {
  let l = 0, sum = 0, minLen = Infinity;
  for (let r = 0; r < nums.length; r++) {
    sum += nums[r];
    while (sum >= target) { minLen = Math.min(minLen, r - l + 1); sum -= nums[l++]; }
  }
  return minLen === Infinity ? 0 : minLen;
}

```

复杂度：时间 $O(n)$ ，空间 $O(1)$

10. 移动零 ●

频度：67 | 难度：简单 | LeetCode 283

题意：把 0 移到末尾，保持非零元素相对顺序。

💡 一句话思路：l 指向下一个非零应放位置，r 遍历遇到非零就交换。

💬 记忆口诀：l 等位，r 找货，非零就换

⚡ 速记模板：

```
function moveZeroes(nums) {
  let l = 0;
  for (let r = 0; r < nums.length; r++) {
    if (nums[r] !== 0) { [nums[l], nums[r]] = [nums[r], nums[l]]; l++; }
  }
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

11. 删除链表的倒数第 N 个节点

频度：181 | 难度：中等 | LeetCode 19

题意：删除链表倒数第 n 个节点。

💡 **一句话思路**：fast 先走 n 步，然后一起走，fast 到尾时 slow 在目标前一个。

🧠 **记忆口诀**：快先走 N 步，一起走到尾，慢的下一个就是要删的

📖 **通俗解释**：用 dummy 处理头节点被删的情况。fast 先跑 n 步，然后两人一起跑，fast 到终点时 slow 正好在目标前一个。

⚡ **速记模板**：

```
function removeNthFromEnd(head, n) {
  const dummy = new ListNode(0, head);
  let fast = dummy, slow = dummy;
  for (let i = 0; i < n; i++) fast = fast.next;
  while (fast.next) { slow = slow.next; fast = fast.next; }
  slow.next = slow.next.next;
  return dummy.next;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

三、链表操作

💡 **万能模板**：链表三板斧 — ① dummy 处理头节点 ② 改指针前保存 next ③ 返回 dummy.next 或新头

🧠 **记忆口诀**：先 dummy，再保存，改指针，最后返回

⚠️ **常见陷阱**：

- 改指针前不保存 next → 链表断开
- 不用 dummy → 头节点被删需特殊处理
- 快慢指针：判断 fast && fast.next，顺序不能反

12. 反转链表 ●

频度：741 | 难度：简单 | LeetCode 206

题意：反转单链表。

💡 **一句话思路**：三指针迭代，存 next、指 prev、prev 前进、curr 前进。

💬 **记忆口诀**：存 next，指 prev，prev 前进，curr 前进

📖 **通俗解释**：像反转火车方向。每节车厢：先记住下一节在哪，把连接反转到前一节，然后都往前走。

⚡ **速记模板**：

```
function reverseList(head) {
  let prev = null, curr = head;
  while (curr) {
    const next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

13. K 个一组翻转链表 ●

频度：514 | 难度：困难 | LeetCode 25

题意：每 k 个节点一组翻转，不足 k 个保持原样。

💡 **一句话思路**：先数够 k 个，翻转这组，接回去，prev 移到组尾。

💬 **记忆口诀**：先数够 K 个，再翻转，接回去，prev 移到组尾

⚡ **速记模板**：

```
function reverseKGroup(head, k) {
  const dummy = new ListNode(0, head);
  let prev = dummy;
  while (true) {
    let tail = prev;
    for (let i = 0; i < k; i++) { tail = tail.next; if (!tail) return dummy.next; }
    const nextGroup = tail.next;
    const reversed = reverse(prev.next, tail);
    const oldHead = prev.next;
    prev.next = reversed;
    oldHead.next = nextGroup;
    prev = oldHead;
  }
}

function reverse(head, tail) {
  let prev = null, curr = head;
  while (curr <= tail) {
    const next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
  }
  return prev;
}
```

```
let prev = null, curr = head;
while (prev !== tail) {
  const next = curr.next; curr.next = prev; prev = curr; curr = next;
}
return tail;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

14. 合并两个有序链表 ●

频度：330 | 难度：简单 | LeetCode 21

题意：合并两个升序链表。

💡 **一句话思路**：dummy + 双指针，谁小接谁，接完谁走。

🧠 **记忆口诀**：谁小接谁，接完谁走

⚡ **速记模板**：

```
function mergeTwoLists(l1, l2) {
  const dummy = new ListNode(-1);
  let curr = dummy;
  while (l1 && l2) {
    if (l1.val < l2.val) { curr.next = l1; l1 = l1.next; }
    else { curr.next = l2; l2 = l2.next; }
    curr = curr.next;
  }
  curr.next = l1 || l2;
  return dummy.next;
}
```

复杂度：时间 $O(m+n)$ ，空间 $O(1)$

15. 反转链表 II ●

频度：267 | 难度：中等 | LeetCode 92

题意：反转从 left 到 right 的节点。

💡 **一句话思路**：走到 left 前一个，用头插法逐个反转。

🧠 **记忆口诀**：走到起点，头插反转

⚡ **速记模板**：

```
function reverseBetween(head, left, right) {
  const dummy = new ListNode(0, head);
  let prev = dummy;
  for (let i = 1; i < left; i++) prev = prev.next;
  let curr = prev.next;
  for (let i = 0; i < right - left; i++) {
    const next = curr.next;
    curr.next = next.next;
    next.next = prev.next;
    prev.next = next;
  }
  return dummy.next;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

16. 重排链表

频度：249 | 难度：中等 | LeetCode 143

题意： $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ 重排。

💡 一句话思路：找中点 $\rightarrow$ 反后半 $\rightarrow$ 交叉合并。

💬 记忆口诀：找中点，反后半，交叉合并

⚡ 速记模板：

```
function reorderList(head) {
  if (!head || !head.next) return;
  let slow = head, fast = head;
  while (fast.next && fast.next.next) { slow = slow.next; fast = fast.next.next; }
  let prev = null, curr = slow.next;
  slow.next = null;
  while (curr) { const next = curr.next; curr.next = prev; prev = curr; curr = next; }
  let l1 = head, l2 = prev;
  while (l2) { const t1 = l1.next, t2 = l2.next; l1.next = l2; l2.next = t1; l1 = t1; l2 = t2; }
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

17. 环形链表

频度：248 | 难度：简单 | LeetCode 141

题意：判断链表是否有环。

💡 一句话思路：快慢指针，快两步慢一步，相遇就有环。

💬 记忆口诀：快两步，慢一步，相遇就有环

⚡ 速记模板:

```
function hasCycle(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next; fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}
```

复杂度: 时间 $O(n)$, 空间 $O(1)$

18. 相交链表 ●

频度: 200 | 难度: 简单 | **LeetCode 160**

题意: 找到两个链表的相交节点。

💡 **一句话思路:** pA 走完 A 走 B, pB 走完 B 走 A, 相遇就是交点。

💡 **记忆口诀:** 你走我的路, 我走你的路, 相遇就是交点

⚡ 速记模板:

```
function getIntersectionNode(headA, headB) {
  let pA = headA, pB = headB;
  while (pA !== pB) {
    pA = pA ? pA.next : headB;
    pB = pB ? pB.next : headA;
  }
  return pA;
}
```

复杂度: 时间 $O(m+n)$, 空间 $O(1)$

19. 删除排序链表中的重复元素 II ●

频度: 184 | 难度: 中等 | **LeetCode 82**

题意: 删除所有重复数字的节点 (一个不留)。

💡 **一句话思路:** dummy + 发现重复就跳过所有相同值。

💡 **记忆口诀:** 遇重复全跳过, 不重复 prev 才前进

⚡ 速记模板:

```
function deleteDuplicates(head) {
  const dummy = new ListNode(0, head);
  let prev = dummy, curr = head;
  while (curr && curr.next) {
    if (curr.val === curr.next.val) {
      while (curr.next && curr.val === curr.next.val) curr = curr.next;
      prev.next = curr.next;
    } else prev = curr;
    curr = curr.next;
  }
  return dummy.next;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

20. 环形链表 II ●

频度：170 | 难度：中等 | LeetCode 142

题意：返回入环第一个节点。

💡 **一句话思路**：相遇后，一个从头一个从相遇点同步走，再遇就是入环点。

💬 **记忆口诀**：相遇后，一头一遇，同步走，再遇就是入环点

⚡ **速记模板**：

```
function detectCycle(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next; fast = fast.next.next;
    if (slow === fast) {
      slow = head;
      while (slow !== fast) { slow = slow.next; fast = fast.next; }
      return slow;
    }
  }
  return null;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

21. 排序链表 ●

频度：146 | 难度：中等 | LeetCode 148

题意： $O(n \log n)$ 排序链表。

💡 **一句话思路**：归并排序：找中点 → 递归排 → 合并。

💬 **记忆口诀**：切两半，递归排，再合并

⚡ 速记模板:

```
function sortList(head) {
  if (!head || !head.next) return head;
  let slow = head, fast = head;
  while (fast.next && fast.next.next) { slow = slow.next; fast = fast.next.next; }
  const right = sortList(slow.next);
  slow.next = null;
  const left = sortList(head);
  return mergeTwoLists(left, right);
}
```

复杂度: 时间 $O(n \log n)$, 空间 $O(\log n)$

22. 回文链表 ●

频度: 85 | 难度: 简单 | LeetCode 234

题意: 判断链表是否回文。

💡 一句话思路: 找中点 → 反后半 → 逐位比。

🧠 记忆口诀: 找中点, 反后半, 逐位比

⚡ 速记模板:

```
function isPalindrome(head) {
  if (!head || !head.next) return true;
  let slow = head, fast = head;
  while (fast.next && fast.next.next) { slow = slow.next; fast = fast.next.next; }
  let prev = null, curr = slow.next;
  while (curr) { const next = curr.next; curr.next = prev; prev = curr; curr = next; }
  let left = head, right = prev;
  while (right) {
    if (left.val !== right.val) return false;
    left = left.next; right = right.next;
  }
  return true;
}
```

复杂度: 时间 $O(n)$, 空间 $O(1)$

23. 两两交换链表中的节点 ●

频度: 70 | 难度: 中等 | LeetCode 24

题意: 两两交换相邻节点。

💡 一句话思路: 递归交换前两个, 第一个连递归结果。

🧠 记忆口诀: 换前俩, 第一个连递归

⚡ 速记模板:

```
function swapPairs(head) {  
  if (!head || !head.next) return head;  
  const first = head, second = head.next;  
  first.next = swapPairs(second.next);  
  second.next = first;  
  return second;  
}
```

复杂度: 时间 $O(n)$, 空间 $O(n)$

四、二叉树遍历与属性

💡 **万能模板:** 树题三板斧 — ① 递归三要素 (终止条件、单层逻辑、返回值) ② 层序用队列 ③ BST 用中序有序

🧠 **记忆口诀:** 当前节点 + 左子树 + 右子树; 空节点是边界

⚠️ **常见陷阱:**

- 递归函数必须有终止条件 (通常是 `!root`)
- 路径问题: 累加和要回溯或传新值
- BST 验证要用范围传递, 不能只比较左右子节点

24. 二叉树的层序遍历 ●

频度: 326 | 难度: 中等 | LeetCode 102

题意: 按层返回节点值。

💡 **一句话思路:** BFS 队列, 每层记录 `queue.length` 控制一层一层处理。

🧠 **记忆口诀:** 队列存节点, 每层定长度

⚡ **速记模板:**

```
function levelOrder(root) {  
  if (!root) return [];  
  const result = [], queue = [root];  
  while (queue.length) {  
    const len = queue.length, level = [];  
    for (let i = 0; i < len; i++) {  
      const node = queue.shift();  
      level.push(node.val);  
      if (node.left) queue.push(node.left);  
      if (node.right) queue.push(node.right);  
    }  
    result.push(level);  
  }  
  return result;  
}
```

```
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

25. 二叉树的锯齿形层次遍历

频度：265 | 难度：中等 | LeetCode 103

题意：Z 字形层序遍历。

💡 **一句话思路**：层序遍历 + 奇数层 reverse 或 unshift。

🧠 **记忆口诀**：层序加方向，奇偶反着来

⚡ **速记模板**：

```
function zigzagLevelOrder(root) {
  if (!root) return [];
  const result = [], queue = [root];
  let leftToRight = true;
  while (queue.length) {
    const len = queue.length, level = [];
    for (let i = 0; i < len; i++) {
      const node = queue.shift();
      leftToRight ? level.push(node.val) : level.unshift(node.val);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    result.push(level);
    leftToRight = !leftToRight;
  }
  return result;
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

26. 二叉树的最近公共祖先

频度：264 | 难度：中等 | LeetCode 236

题意：找到两个节点的最近公共祖先。

💡 **一句话思路**：递归，左右各找，都有就当前是 LCA，只有一边就返回那一边。

🧠 **记忆口诀**：左右各找，都有返回当前，一边返回那一边

📖 **通俗解释**：如果 p 和 q 分别在左右子树，当前节点就是 LCA。如果都在一边，LCA 就在那边。

⚡ **速记模板**：

```
function lowestCommonAncestor(root, p, q) {
  if (!root || root === p || root === q) return root;
  const left = lowestCommonAncestor(root.left, p, q);
  const right = lowestCommonAncestor(root.right, p, q);
  if (left && right) return root;
  return left || right;
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

27. 二叉树的右视图 ●

频度：160 | 难度：中等 | LeetCode 199

题意：返回从右侧看到的节点值。

💡 **一句话思路**：层序遍历，每层取最后一个节点。

💡 **记忆口诀**：层序遍历，取每层最后一个

⚡ **速记模板**：

```
function rightSideview(root) {
  if (!root) return [];
  const result = [], queue = [root];
  while (queue.length) {
    const len = queue.length;
    let last;
    for (let i = 0; i < len; i++) {
      const node = queue.shift();
      last = node.val;
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    result.push(last);
  }
  return result;
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

28. 二叉树的中序遍历 ●

频度：142 | 难度：简单 | LeetCode 94

题意：中序遍历（左-根-右）。

💡 **一句话思路**：迭代用栈，一直往左走到底，弹出后往右走。

💡 **记忆口诀**：一路向左，弹出访问，转向右边

⚡ **速记模板**：

```
function inorderTraversal(root) {
  const result = [], stack = [];
  let curr = root;
  while (curr || stack.length) {
    while (curr) { stack.push(curr); curr = curr.left; }
    curr = stack.pop();
    result.push(curr.val);
    curr = curr.right;
  }
  return result;
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

29. 从前序与中序遍历序列构造二叉树 ●

频度：114 | 难度：中等 | LeetCode 105

题意：根据前序和中序构造二叉树。

💡 **一句话思路**：前序第一个是根，在中序中找到根的位置划分左右子树。

💬 **记忆口诀**：前序定根，中序分左右

📖 **通俗解释**：前序遍历的第一个元素一定是根节点。在中序遍历中找到这个根，左边是左子树，右边是右子树。然后递归构造。

⚡ **速记模板**：

```
function buildTree(preorder, inorder) {
  if (!preorder.length) return null;
  const val = preorder[0];
  const root = new TreeNode(val);
  const idx = inorder.indexOf(val);
  root.left = buildTree(preorder.slice(1, idx+1), inorder.slice(0, idx));
  root.right = buildTree(preorder.slice(idx+1), inorder.slice(idx+1));
  return root;
}
```

复杂度：时间 $O(n^2)$ ，空间 $O(n)$

30. 对称二叉树 ●

频度：96 | 难度：简单 | LeetCode 101

题意：判断二叉树是否镜像对称。

💡 **一句话思路**：递归比较左子树的左和右子树的右，左子树的右和右子树的左。

💬 **记忆口诀**：左左比右右，左右比右左

⚡ **速记模板**：

```
function isSymmetric(root) {
  if (!root) return true;
  return isMirror(root.left, root.right);
}
function isMirror(l, r) {
  if (!l && !r) return true;
  if (!l || !r) return false;
  return l.val === r.val && isMirror(l.left, r.right) && isMirror(l.right, r.left);
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

31. 二叉树的最大深度 ●

频度：91 | 难度：简单 | LeetCode 104

题意：二叉树的最大深度。

💡 一句话思路：递归， $\max(\text{左深度}, \text{右深度}) + 1$ 。

💬 记忆口诀：左右取大加一

⚡ 速记模板：

```
function maxDepth(root) {
  if (!root) return 0;
  return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1;
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

32. 平衡二叉树 ●

频度：87 | 难度：简单 | LeetCode 110

题意：判断是否是平衡二叉树（左右子树高度差不超过 1）。

💡 一句话思路：自底向上，用 -1 表示不平衡，避免重复计算高度。

💬 记忆口诀：-1 表示不平衡，左右都非负才比差

⚡ 速记模板：

```
function isBalanced(root) {
    return height(root) !== -1;
}
function height(root) {
    if (!root) return 0;
    const l = height(root.left);
    if (l === -1) return -1;
    const r = height(root.right);
    if (r === -1) return -1;
    if (Math.abs(l - r) > 1) return -1;
    return Math.max(l, r) + 1;
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

33. 验证二叉搜索树 ●

频度：84 | 难度：中等 | LeetCode 98

题意：验证是否是有效的 BST。

💡 **一句话思路**：中序遍历应该严格递增，或递归传递上下界。

💬 **记忆口诀**：中序递增即 BST

📖 **通俗解释**：BST 的中序遍历结果一定是严格递增的。用一个 prev 变量记录上一个访问的值，如果当前值 $\leq$ prev 就不是 BST。

⚡ **速记模板**：

```
function isValidBST(root) {
    let prev = -Infinity;
    function inorder(node) {
        if (!node) return true;
        if (!inorder(node.left)) return false;
        if (node.val <= prev) return false;
        prev = node.val;
        return inorder(node.right);
    }
    return inorder(root);
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

34. 二叉树的前序遍历 ●

频度：84 | 难度：简单 | LeetCode 144

题意：前序遍历（根-左-右）。

💡 **一句话思路**：迭代用栈，先压右再压左（因为栈是后进先出）。

🧠 记忆口诀：栈存根，先右后左

⚡ 速记模板：

```
function preorderTraversal(root) {
  if (!root) return [];
  const result = [], stack = [root];
  while (stack.length) {
    const node = stack.pop();
    result.push(node.val);
    if (node.right) stack.push(node.right);
    if (node.left) stack.push(node.left);
  }
  return result;
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

35. 二叉树的最大宽度 🟡

频度：82 | 难度：中等 | LeetCode 662

题意：二叉树的最大宽度（含 null 的层宽度）。

💡 一句话思路：BFS 给每个节点编号，根为 0，左子 $2id$ ，右子 $2id+1$ 。

🧠 记忆口诀：BFS 编号，每层首尾差加一

⚡ 速记模板：

```
function widthOfBinaryTree(root) {
  if (!root) return 0;
  const queue = [[root, 0n]];
  let maxWidth = 0n;
  while (queue.length) {
    const len = queue.length;
    const leftIdx = queue[0][1];
    for (let i = 0; i < len; i++) {
      const [node, idx] = queue.shift();
      if (i === len - 1) maxWidth = Math.max(maxWidth, idx - leftIdx + 1n);
      if (node.left) queue.push([node.left, idx * 2n]);
      if (node.right) queue.push([node.right, idx * 2n + 1n]);
    }
  }
  return Number(maxWidth);
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

36. 二叉树的直径 ●

频度：81 | 难度：简单 | LeetCode 543

题意：二叉树中最长路径的边数。

💡 一句话思路：每个节点的直径 = 左深度 + 右深度，全局取最大。

💡 记忆口诀：直径 = 左深 + 右深，全局取最大

⚡ 速记模板：

```
function diameterOfBinaryTree(root) {  
  let diameter = 0;  
  function depth(node) {  
    if (!node) return 0;  
    const l = depth(node.left), r = depth(node.right);  
    diameter = Math.max(diameter, l + r);  
    return Math.max(l, r) + 1;  
  }  
  depth(root);  
  return diameter;  
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

37. 翻转二叉树 ●

频度：68 | 难度：简单 | LeetCode 226

题意：翻转二叉树（左右交换）。

💡 一句话思路：递归交换左右子树。

💡 记忆口诀：交换左右，递归下去

⚡ 速记模板：

```
function invertTree(root) {  
  if (!root) return null;  
  [root.left, root.right] = [root.right, root.left];  
  invertTree(root.left);  
  invertTree(root.right);  
  return root;  
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

38. 路径总和 ●

频度：72 | 难度：简单 | LeetCode 112

题意：是否存在根到叶子路径和等于 targetSum。

💡 一句话思路：递归减去当前值，到叶子判断是否为 0。

💬 记忆口诀：一路减，到叶判零

⚡ 速记模板：

```
function hasPathSum(root, targetSum) {  
  if (!root) return false;  
  targetSum -= root.val;  
  if (!root.left && !root.right) return targetSum === 0;  
  return hasPathSum(root.left, targetSum) || hasPathSum(root.right, targetSum);  
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

39. 路径总和 II ●

频度：77 | 难度：中等 | LeetCode 113

题意：返回所有根到叶子路径和等于 targetSum 的路径。

💡 一句话思路：DFS + 回溯，到叶子满足条件就记录路径。

💬 记忆口诀：DFS 加回溯，到叶满足就记录

⚡ 速记模板：

```
function pathSum(root, targetSum) {  
  const result = [];  
  function dfs(node, remain, path) {  
    if (!node) return;  
    path.push(node.val);  
    if (!node.left && !node.right && remain === node.val) result.push([...path]);  
    else { dfs(node.left, remain - node.val, path); dfs(node.right, remain - node.val,  
path); }  
    path.pop();  
  }  
  dfs(root, targetSum, []);  
  return result;  
}
```

复杂度：时间 $O(n^2)$ ，空间 $O(n)$

40. 求根到叶子节点数字之和 ●

频度：100 | 难度：中等 | LeetCode 129

题意：根到叶子每条路径组成一个数字，求所有数字之和。

💡 **一句话思路**：DFS 传递当前累积值 $\text{curr} = \text{curr} * 10 + \text{node.val}$ 。

🧠 **记忆口诀**：curr 乘十加当前，到叶累加

⚡ **速记模板**：

```
function sumNumbers(root) {  
  let total = 0;  
  function dfs(node, curr) {  
    if (!node) return;  
    curr = curr * 10 + node.val;  
    if (!node.left && !node.right) { total += curr; return; }  
    dfs(node.left, curr);  
    dfs(node.right, curr);  
  }  
  dfs(root, 0);  
  return total;  
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

41. 二叉树中的最大路径和 ●

频度：182 | 难度：困难 | LeetCode 124

题意：任意节点到任意节点的最大路径和。

💡 **一句话思路**：每个节点贡献 = 左正贡献 + 右正贡献 + 节点值，全局取最大。

🧠 **记忆口诀**：左右取正加自身，全局更新最大值

📄 **通俗解释**：对每个节点，计算"经过该节点的最大路径和" = $\max(0, \text{左子树贡献}) + \max(0, \text{右子树贡献}) + \text{节点值}$ 。递归返回"单边最大贡献"给父节点。

⚡ **速记模板**：

```
function maxPathSum(root) {
  let maxSum = -Infinity;
  function dfs(node) {
    if (!node) return 0;
    const l = Math.max(dfs(node.left), 0);
    const r = Math.max(dfs(node.right), 0);
    maxSum = Math.max(maxSum, l + r + node.val);
    return node.val + Math.max(l, r);
  }
  dfs(root);
  return maxSum;
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

五、动态规划

💡 **万能模板**：DP 四步法 — ① 定义 dp 含义 ② 初始化边界 ③ 写转移方程 ④ 确定返回值

🧠 **记忆口诀**：定义状态、初始化、找转移、看优化

⚠️ **常见陷阱**：

- dp 数组大小通常是 $n+1$ （多一个边界位）
- 初始化第一行/第一列/第一个元素
- 能滚动优化空间时，注意更新顺序（从后往前）

42. 最大子数组和 ●

频度：371 | 难度：中等 | LeetCode 53

题意：和最大的连续子数组。

💡 **一句话思路**：Kadane 算法， $\text{curr} = \max(\text{当前数}, \text{curr} + \text{当前数})$ 。

🧠 **记忆口诀**：要么自己重新开始，要么接着前面干

📖 **通俗解释**：走到每个位置，决定是"从自己重新开始"还是"接着前面的子数组"。选大的那个，同时更新全局最大。

⚡ **速记模板**：

```
function maxSubArray(nums) {
  let curr = nums[0], max = nums[0];
  for (let i = 1; i < nums.length; i++) {
    curr = Math.max(nums[i], curr + nums[i]);
    max = Math.max(max, curr);
  }
  return max;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

43. 最长上升子序列 ●

频度：262 | 难度：中等 | LeetCode 300

题意：最长严格递增子序列长度。

💡 **一句话思路**：贪心 + 二分，tails 数组维护每个长度的最小末尾。

🧠 **记忆口诀**：tails 存最小末尾，二分找插入位

📖 **通俗解释**：tails[i] 表示长度为 i+1 的递增子序列的最小末尾。遍历每个数，用二分找到它在 tails 中的位置（第一个 $\geq$ 它的），替换掉。tails 的长度就是答案。

⚡ **速记模板**：

```
function lengthOfLIS(nums) {
  const tails = [];
  for (const num of nums) {
    let l = 0, r = tails.length;
    while (l < r) {
      const mid = (l + r) >> 1;
      tails[mid] < num ? l = mid + 1 : r = mid;
    }
    tails[l] = num;
  }
  return tails.length;
}
```

复杂度：时间 $O(n \log n)$ ，空间 $O(n)$

44. 编辑距离 ●

频度：199 | 难度：困难 | LeetCode 72

题意：word1 转 word2 的最小操作数（插入/删除/替换）。

💡 **一句话思路**：dp[i][j] = word1 前 i 个转 word2 前 j 个的最小操作数。

🧠 **记忆口诀**：相等继承左上，不等三选一最小加一

📖 **通俗解释**：dp[i][j] 表示把 word1 前 i 个字符变成 word2 前 j 个字符需要的最少操作。如果字符相同，直接继承 dp[i-1][j-1]；否则取插入、删除、替换三种操作的最小值加 1。

⚡ **速记模板**：

```
function minDistance(word1, word2) {
  const m = word1.length, n = word2.length;
  const dp = Array.from({length: m+1}, () => new Array(n+1).fill(0));
  for (let i = 0; i <= m; i++) dp[i][0] = i;
  for (let j = 0; j <= n; j++) dp[0][j] = j;
  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (word1[i-1] === word2[j-1]) dp[i][j] = dp[i-1][j-1];
      else dp[i][j] = 1 + Math.min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
    }
  }
  return dp[m][n];
}
```

复杂度：时间 $O(mn)$ ，空间 $O(mn)$

45. 最长公共子序列 ●

频度：193 | 难度：中等 | LeetCode 1143

题意：两个字符串的最长公共子序列长度。

💡 一句话思路：dp[i][j]，相同就左上+1，不同就取上左最大。

💬 记忆口诀：相同左上加一，不同上左取大

⚡ 速记模板：

```
function longestCommonSubsequence(text1, text2) {
  const m = text1.length, n = text2.length;
  const dp = Array.from({length: m+1}, () => new Array(n+1).fill(0));
  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (text1[i-1] === text2[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
      else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
    }
  }
  return dp[m][n];
}
```

复杂度：时间 $O(mn)$ ，空间 $O(mn)$

46. 最长有效括号 ●

频度：146 | 难度：困难 | LeetCode 32

题意：最长有效括号子串长度。

💡 一句话思路：dp[i] 表示以 i 结尾的最长有效括号长度。

💬 记忆口诀：遇到) 往前看，配对就加二再加前一个

⚡ 速记模板:

```
function longestValidParentheses(s) {
  const dp = new Array(s.length).fill(0);
  let max = 0;
  for (let i = 1; i < s.length; i++) {
    if (s[i] === ')') {
      if (s[i-1] === '(') dp[i] = (i >= 2 ? dp[i-2] : 0) + 2;
      else if (i - dp[i-1] - 1 >= 0 && s[i-dp[i-1]-1] === '(')
        dp[i] = dp[i-1] + 2 + (i-dp[i-1]-2 >= 0 ? dp[i-dp[i-1]-2] : 0);
      max = Math.max(max, dp[i]);
    }
  }
  return max;
}
```

复杂度: 时间 $O(n)$, 空间 $O(n)$

47. 零钱兑换 ●

频度: 132 | 难度: 中等 | LeetCode 322

题意: 凑成 amount 的最少硬币数。

💡 一句话思路: $dp[i] = \min(dp[i], dp[i - \text{coin}] + 1)$ 。

💡 记忆口诀: 每个金额试所有硬币, 取最小加一

⚡ 速记模板:

```
function coinChange(coins, amount) {
  const dp = new Array(amount + 1).fill(Infinity);
  dp[0] = 0;
  for (let i = 1; i <= amount; i++) {
    for (const coin of coins) {
      if (coin <= i) dp[i] = Math.min(dp[i], dp[i - coin] + 1);
    }
  }
  return dp[amount] === Infinity ? -1 : dp[amount];
}
```

复杂度: 时间 $O(\text{amount} \times \text{coins.length})$, 空间 $O(\text{amount})$

48. 爬楼梯 ●

频度: 128 | 难度: 简单 | LeetCode 70

题意: 每次爬 1 或 2 阶, n 阶有多少种方法。

💡 一句话思路: 斐波那契数列, $dp[i] = dp[i-1] + dp[i-2]$ 。

💡 记忆口诀: 到第 i 阶 = 从 $i-1$ 爬 1 步 + 从 $i-2$ 爬 2 步

⚡ 速记模板:

```
function climbStairs(n) {  
  if (n <= 2) return n;  
  let a = 1, b = 2;  
  for (let i = 3; i <= n; i++) { const c = a + b; a = b; b = c; }  
  return b;  
}
```

复杂度: 时间 $O(n)$, 空间 $O(1)$

49. 乘积最大子数组 ●

频度: 83 | 难度: 中等 | LeetCode 152

题意: 乘积最大的连续子数组。

💡 **一句话思路:** 同时维护最大值和最小值 (负数会让最小变最大)。

💡 **记忆口诀:** 最大最小一起维护, 负数来了就交换

📖 **通俗解释:** 因为有负数, 最小的乘积乘以负数可能变成最大。所以每次都要记录当前的最大和最小, 遇到负数时先交换它们再计算。

⚡ 速记模板:

```
function maxProduct(nums) {  
  let max = nums[0], min = nums[0], res = nums[0];  
  for (let i = 1; i < nums.length; i++) {  
    const prevMax = max;  
    max = Math.max(nums[i], prevMax * nums[i], min * nums[i]);  
    min = Math.min(nums[i], prevMax * nums[i], min * nums[i]);  
    res = Math.max(res, max);  
  }  
  return res;  
}
```

复杂度: 时间 $O(n)$, 空间 $O(1)$

50. 不同路径 ●

频度: 77 | 难度: 中等 | LeetCode 62

题意: $m \times n$ 网格从左上到右下, 只能向右或向下, 多少条路。

💡 **一句话思路:** $dp[i][j] = dp[i-1][j] + dp[i][j-1]$, 可优化为一维数组。

💡 **记忆口诀:** 每格 = 上面来 + 左边来

⚡ 速记模板:

```
function uniquePaths(m, n) {  
  const dp = new Array(n).fill(1);  
  for (let i = 1; i < m; i++)  
    for (let j = 1; j < n; j++)  
      dp[j] += dp[j - 1];  
  return dp[n - 1];  
}
```

复杂度：时间 $O(mn)$ ，空间 $O(n)$

51. 打家劫舍 ●

频度：73 | 难度：中等 | LeetCode 198

题意：相邻房屋不能同时偷，求最大金额。

💡 **一句话思路**： $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$ 。

🧠 **记忆口诀**：要么不偷当前，要么偷当前加前前

⚡ **速记模板**：

```
function rob(nums) {  
  let prev2 = 0, prev1 = 0;  
  for (const num of nums) {  
    const curr = Math.max(prev1, prev2 + num);  
    prev2 = prev1; prev1 = curr;  
  }  
  return prev1;  
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

52. 最小路径和 ●

频度：94 | 难度：中等 | LeetCode 64

题意：左上到右下路径数字和最小。

💡 **一句话思路**：原地 DP，每格加上上方和左方的较小值。

🧠 **记忆口诀**：每格 = 自身 + $\min(\text{上}, \text{左})$

⚡ **速记模板**：

```
function minPathSum(grid) {
  const m = grid.length, n = grid[0].length;
  for (let i = 1; i < m; i++) grid[i][0] += grid[i-1][0];
  for (let j = 1; j < n; j++) grid[0][j] += grid[0][j-1];
  for (let i = 1; i < m; i++)
    for (let j = 1; j < n; j++)
      grid[i][j] += Math.min(grid[i-1][j], grid[i][j-1]);
  return grid[m-1][n-1];
}
```

复杂度：时间 $O(mn)$ ，空间 $O(1)$

53. 最大正方形

频度： 88 | **难度：** 中等 | **LeetCode 221**

题意： 只含 1 的矩阵中最大正方形面积。

💡 **一句话思路：** $dp[i][j] = \min(\text{上}, \text{左}, \text{左上}) + 1$ （当前为 1 时）。

💬 **记忆口诀：** 当前为 1，取三方向最小加一

⚡ **速记模板：**

```
function maximalSquare(matrix) {
  const m = matrix.length, n = matrix[0].length;
  const dp = Array.from({length: m}, () => new Array(n).fill(0));
  let maxSide = 0;
  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      if (matrix[i][j] === '1') {
        dp[i][j] = (i === 0 || j === 0) ? 1 : Math.min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1;
        maxSide = Math.max(maxSide, dp[i][j]);
      }
    }
  }
  return maxSide * maxSide;
}
```

复杂度：时间 $O(mn)$ ，空间 $O(mn)$

六、字符串处理

💡 **万能模板：** 字符串题 = 按字符扫描 + 栈/窗口/双指针/状态机保存上下文

💬 **记忆口诀：** 逐字符处理，栈管嵌套，窗口管子串

⚠️ **常见陷阱：**

- 括号题：栈空时遇到右括号直接返回 false

- 大数运算：不能直接转 Number，要逐位计算
- 字符串比较：注意前导零和长度不同

54. 最长回文子串 ●

频度：346 | 难度：中等 | LeetCode 5

题意：最长回文子串。

💡 **一句话思路**：中心扩展法，每个字符和字符间隙都作为中心向两边扩展。

💡 **记忆口诀**：每个中心向两边，奇偶都要试

⚡ **速记模板**：

```
function longestPalindrome(s) {
  let start = 0, maxLen = 1;
  function expand(l, r) {
    while (l >= 0 && r < s.length && s[l] === s[r]) { l--; r++; }
    const len = r - l - 1;
    if (len > maxLen) { maxLen = len; start = l + 1; }
  }
  for (let i = 0; i < s.length; i++) { expand(i, i); expand(i, i + 1); }
  return s.substring(start, start + maxLen);
}
```

复杂度：时间 $O(n^2)$ ，空间 $O(1)$

55. 有效的括号 ●

频度：292 | 难度：简单 | LeetCode 20

题意：判断括号字符串是否有效。

💡 **一句话思路**：左括号入栈，右括号检查栈顶是否匹配。

💡 **记忆口诀**：左进栈，右匹配，最后栈要空

⚡ **速记模板**：

```
function isValid(s) {
  const stack = [];
  const map = {'(': ')', '[': ']', '{': '}'};
  for (const c of s) {
    if (c === '(' || c === '[' || c === '{') stack.push(c);
    else if (stack.pop() !== map[c]) return false;
  }
  return stack.length === 0;
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

56. 字符串相加 ●

频度：241 | 难度：简单 | LeetCode 415

题意：字符串形式的非负整数相加。

💡 一句话思路：从后往前逐位相加，维护进位。

🧠 记忆口诀：从后往前加，进位跟着走

⚡ 速记模板：

```
function addStrings(num1, num2) {
  let i = num1.length-1, j = num2.length-1, carry = 0;
  const res = [];
  while (i >= 0 || j >= 0 || carry) {
    const sum = (+num1[i--] || 0) + (+num2[j--] || 0) + carry;
    res.push(sum % 10);
    carry = Math.floor(sum / 10);
  }
  return res.reverse().join('');
}
```

复杂度：时间 $O(\max(m,n))$ ，空间 $O(\max(m,n))$

57. 复原 IP 地址 ●

频度：185 | 难度：中等 | LeetCode 93

题意：给数字字符串，返回所有可能的有效 IP 地址。

💡 一句话思路：回溯，每段 1-3 位，值 0-255，不能有前导零。

🧠 记忆口诀：回溯分段，1-3 位，0-255，无前导零

⚡ 速记模板：

```
function restoreIpAddresses(s) {
  const result = [];
  function backtrack(start, path) {
    if (path.length === 4) { if (start === s.length) result.push(path.join('.')); return; }
    for (let len = 1; len <= 3; len++) {
      const seg = s.substr(start, len);
      if (len > 1 && seg[0] === '0') break;
      if (+seg <= 255) { path.push(seg); backtrack(start + len, path); path.pop(); }
    }
  }
  backtrack(0, []);
  return result;
}
```

复杂度：时间 $O(1)$ ，空间 $O(1)$

58. 比较版本号

频度：160 | 难度：中等 | LeetCode 165

题意：比较两个版本号大小。

💡 一句话思路：按 . 分割，逐段比较，缺的补 0。

💬 记忆口诀：分割逐段比，缺的当零

⚡ 速记模板：

```
function compareVersion(version1, version2) {
  const v1 = version1.split('.'), v2 = version2.split('.');
  const n = Math.max(v1.length, v2.length);
  for (let i = 0; i < n; i++) {
    const a = +(v1[i] || 0), b = +(v2[i] || 0);
    if (a > b) return 1; if (a < b) return -1;
  }
  return 0;
}
```

复杂度：时间 $O(m+n)$ ，空间 $O(m+n)$

59. 字符串转换整数 atoi

频度：131 | 难度：中等 | LeetCode 8

题意：实现 atoi，处理空格、符号、数字、溢出。

💡 一句话思路：跳过空格 → 处理符号 → 逐位累加 → 检查溢出。

💬 记忆口诀：空、符、数、溢，四步走

⚡ 速记模板：

```
function myAtoi(s) {
  const INT_MAX = 2147483647, INT_MIN = -2147483648;
  let i = 0, sign = 1, result = 0;
  while (s[i] === ' ') i++;
  if (s[i] === '+' || s[i] === '-') sign = s[i++] === '-' ? -1 : 1;
  while (i < s.length && s[i] >= '0' && s[i] <= '9') {
    const digit = +s[i];
    if (result > (INT_MAX - digit) / 10) return sign === 1 ? INT_MAX : INT_MIN;
    result = result * 10 + digit; i++;
  }
  return sign * result;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

60. 字符串相乘 ●

频度：130 | 难度：中等 | LeetCode 43

题意：字符串形式的非负整数相乘。

💡 一句话思路：模拟竖式乘法，pos 数组存储每位结果。

🧠 记忆口诀：竖式乘法，pos[i+j] 和 pos[i+j+1] 存结果

⚡ 速记模板：

```
function multiply(num1, num2) {
  if (num1 === '0' || num2 === '0') return '0';
  const m = num1.length, n = num2.length;
  const pos = new Array(m + n).fill(0);
  for (let i = m-1; i >= 0; i--) {
    for (let j = n-1; j >= 0; j--) {
      const prod = (num1[i]-'0') * (num2[j]-'0');
      const sum = prod + pos[i+j+1];
      pos[i+j] += Math.floor(sum / 10);
      pos[i+j+1] = sum % 10;
    }
  }
  let res = pos.join('');
  while (res[0] === '0') res = res.slice(1);
  return res;
}
```

复杂度：时间 $O(mn)$ ，空间 $O(m+n)$

61. 翻转字符串里的单词 ●

频度：105 | 难度：中等 | LeetCode 151

题意：翻转单词顺序，去除多余空格。

💡 一句话思路：trim + split 正则 + reverse + join。

🧠 记忆口诀：去空、分割、反转、拼接

⚡ 速记模板：

```
function reverseWords(s) {
  return s.trim().split(/\s+/).reverse().join(' ');
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

62. 最长公共前缀 ●

频度：83 | 难度：简单 | LeetCode 14

题意：字符串数组的最长公共前缀。

💡 **一句话思路**：以第一个字符串为基准，逐字符比较其他字符串。

🧠 **记忆口诀**：拿第一个当基准，逐位比

⚡ **速记模板**：

```
function longestCommonPrefix(strs) {
  if (!strs.length) return '';
  for (let i = 0; i < strs[0].length; i++) {
    const c = strs[0][i];
    for (let j = 1; j < strs.length; j++) {
      if (i >= strs[j].length || strs[j][i] !== c) return strs[0].slice(0, i);
    }
  }
  return strs[0];
}
```

复杂度：时间 $O(mn)$ ，空间 $O(1)$

63. 括号生成 ●

频度：150 | 难度：中等 | LeetCode 22

题意：生成 n 对括号的所有有效组合。

💡 **一句话思路**：回溯，左括号不够 n 就加左，右括号少于左就加右。

🧠 **记忆口诀**：左不够加左，右少于左加右

⚡ **速记模板**：

```
function generateParenthesis(n) {
  const result = [];
  function backtrack(str, left, right) {
    if (str.length === 2 * n) { result.push(str); return; }
    if (left < n) backtrack(str + '(', left + 1, right);
    if (right < left) backtrack(str + ')', left, right + 1);
  }
  backtrack('', 0, 0);
  return result;
}
```

复杂度：时间 $O(4^n/\sqrt{n})$ ，空间 $O(n)$

64. 字符串解码 ●

频度：97 | 难度：中等 | LeetCode 394

题意：3[a]2[bc] → aaabcbc。

💡 一句话思路：双栈：数字栈和字符串栈，遇到] 就弹出重复。

💬 记忆口诀：数字栈存次数，字符串栈存前缀，遇] 弹出资深

⚡ 速记模板：

```
function decodeString(s) {
  const numStack = [], strStack = [];
  let currNum = 0, currStr = '';
  for (const c of s) {
    if (c >= '0' && c <= '9') currNum = currNum * 10 + +c;
    else if (c === '[') { numStack.push(currNum); strStack.push(currStr); currNum = 0;
currStr = ''; }
    else if (c === ']') currStr = strStack.pop() + currStr.repeat(numStack.pop());
    else currStr += c;
  }
  return currStr;
}
```

复杂度：时间 O(n)，空间 O(n)

65. 基本计算器 II ●

频度：69 | 难度：中等 | LeetCode 227

题意：实现包含 + - * / 的计算器（无括号）。

💡 一句话思路：栈存数字，遇到 * / 立即计算，+ - 存正负数。

💬 记忆口诀：乘除立即算，加减存栈里，最后求和

⚡ 速记模板：

```
function calculate(s) {
  const stack = [];
  let num = 0, sign = '+';
  for (let i = 0; i <= s.length; i++) {
    const c = s[i];
    if (c >= '0' && c <= '9') num = num * 10 + +c;
    else if (c === '+' || c === '-' || c === '*' || c === '/' || i === s.length) {
      if (sign === '+') stack.push(num);
      else if (sign === '-') stack.push(-num);
      else if (sign === '*') stack.push(stack.pop() * num);
      else if (sign === '/') stack.push(parseInt(stack.pop() / num));
      sign = c; num = 0;
    }
  }
  return stack.reduce((a, b) => a + b, 0);
}
```

```
}
```

复杂度：时间 $O(n)$ ，空间 $O(n)$

七、二分查找

💡 **万能模板**：二分 = 利用单调性丢弃一半。先明确搜索的是值、下标、边界还是答案空间。

🧠 **记忆口诀**：左闭右开或双闭，mid 计算防溢出，边界更新要配套

⚠️ **常见陷阱**：

- 循环条件： $l \leq r$ （双闭）或 $l < r$ （左闭右开）
- $mid = (l + r) >> 1$ 或 $l + (r - l) / 2$ 防溢出
- 搜索边界时不要找到就返回，要记录答案继续收缩

66. 搜索旋转排序数组 ●

频度：305 | 难度：中等 | LeetCode 33

题意：旋转数组中找目标值。

💡 **一句话思路**：判断哪侧有序，再判断目标是否在有序侧。

🧠 **记忆口诀**：看哪侧有序，目标在有序侧就缩那边

⚡ **速记模板**：

```
function search(nums, target) {
  let l = 0, r = nums.length - 1;
  while (l <= r) {
    const mid = (l + r) >> 1;
    if (nums[mid] === target) return mid;
    if (nums[l] <= nums[mid]) {
      if (nums[l] <= target && target < nums[mid]) r = mid - 1;
      else l = mid + 1;
    } else {
      if (nums[mid] < target && target <= nums[r]) l = mid + 1;
      else r = mid - 1;
    }
  }
  return -1;
}
```

复杂度：时间 $O(\log n)$ ，空间 $O(1)$

67. 二分查找 ●

频度： 150 | 难度： 简单 | LeetCode 704

题意：有序数组中找目标值。

💡 **一句话思路**：标准二分，mid 等于目标就返回，小于目标缩左，大于目标缩右。

💬 **记忆口诀**：中点比，小缩左，大缩右

⚡ **速记模板**：

```
function search(nums, target) {
  let l = 0, r = nums.length - 1;
  while (l <= r) {
    const mid = (l + r) >> 1;
    if (nums[mid] === target) return mid;
    nums[mid] < target ? l = mid + 1 : r = mid - 1;
  }
  return -1;
}
```

复杂度：时间 $O(\log n)$ ，空间 $O(1)$

68. x 的平方根 ●

频度： 146 | 难度： 简单 | LeetCode 69

题意：返回 $\text{sqrt}(x)$ 的整数部分。

💡 **一句话思路**：二分答案，找最大的 mid 使 $\text{mid} * \text{mid} \leq x$ 。

💬 **记忆口诀**：二分找最大平方不超过 x 的数

⚡ **速记模板**：

```
function mySqrt(x) {
  let l = 0, r = x, ans = 0;
  while (l <= r) {
    const mid = (l + r) >> 1;
    if (mid * mid <= x) { ans = mid; l = mid + 1; }
    else r = mid - 1;
  }
  return ans;
}
```

复杂度：时间 $O(\log x)$ ，空间 $O(1)$

69. 寻找两个正序数组的中位数 ●

频度：169 | 难度：困难 | LeetCode 4

题意：两个有序数组的中位数。

💡 **一句话思路**：在较短数组上二分划分点，保证左半部分都小于右半部分。

🧠 **记忆口诀**：短数组二分，划分点满足交叉小于等于

⚡ **速记模板**：

```
function findMedianSortedArrays(nums1, nums2) {
  if (nums1.length > nums2.length) [nums1, nums2] = [nums2, nums1];
  const m = nums1.length, n = nums2.length;
  let l = 0, r = m;
  while (l <= r) {
    const i = (l + r) >> 1;
    const j = ((m + n + 1) >> 1) - i;
    const maxL1 = i === 0 ? -Infinity : nums1[i-1];
    const minR1 = i === m ? Infinity : nums1[i];
    const maxL2 = j === 0 ? -Infinity : nums2[j-1];
    const minR2 = j === n ? Infinity : nums2[j];
    if (maxL1 <= minR2 && maxL2 <= minR1) {
      if ((m + n) % 2 === 0) return (Math.max(maxL1, maxL2) + Math.min(minR1, minR2)) / 2;
      return Math.max(maxL1, maxL2);
    } else if (maxL1 > minR2) r = i - 1;
    else l = i + 1;
  }
}
```

复杂度：时间 $O(\log(\min(m,n)))$ ，空间 $O(1)$

70. 查找元素首尾位置 ●

频度：102 | 难度：中等 | LeetCode 34

题意：有序数组中找目标值的第一个和最后一个位置。

💡 **一句话思路**：两次二分，一次找左边界，一次找右边界。

🧠 **记忆口诀**：二分找左界，二分找右界

⚡ **速记模板**：

```
function searchRange(nums, target) {
  function binarySearch(left) {
    let l = 0, r = nums.length;
    while (l < r) {
      const mid = (l + r) >> 1;
      if (nums[mid] > target || (left && nums[mid] === target)) r = mid;
      else l = mid + 1;
    }
    return l;
  }
  return [binarySearch(true), binarySearch(false)];
}
```

```

    }
    const first = binarySearch(true);
    if (first === nums.length || nums[first] !== target) return [-1, -1];
    return [first, binarySearch(false) - 1];
}

```

复杂度：时间 $O(\log n)$ ，空间 $O(1)$

71. 寻找峰值 ●

频度：80 | 难度：中等 | LeetCode 162

题意：找峰值元素（大于相邻元素）。

💡 **一句话思路**：二分， $\text{nums}[\text{mid}] > \text{nums}[\text{mid}+1]$ 说明峰值在左（含 mid），否则在右。

🧠 **记忆口诀**：上坡往右走，下坡往左找

⚡ **速记模板**：

```

function findPeakElement(nums) {
    let l = 0, r = nums.length - 1;
    while (l < r) {
        const mid = (l + r) >> 1;
        nums[mid] > nums[mid+1] ? r = mid : l = mid + 1;
    }
    return l;
}

```

复杂度：时间 $O(\log n)$ ，空间 $O(1)$

八、栈与队列

💡 **万能模板**：栈 = LIFO（后进先出），队列 = FIFO（先进先出）。辅助栈用于 $O(1)$ 查询最值。

🧠 **记忆口诀**：栈管最近，队列管顺序，双栈可模拟队列

⚠️ **常见陷阱**：

- 用栈模拟队列：in 栈负责 push，out 栈负责 pop，out 空时才从 in 倒过来
- 单调队列：队头存最大值索引，新元素入队前弹出比它小的

72. 有效的括号 ●（见 55 题）

73. 用栈实现队列 ●

频度：143 | 难度：简单 | LeetCode 232

题意：用两个栈实现队列。

💡 **一句话思路**：in 栈存新元素，out 栈提供出队，out 空时从 in 倒过来。

🧠 记忆口诀：in 管进，out 管出，out 空就倒

⚡ 速记模板：

```
class MyQueue {
    constructor() { this.in = []; this.out = []; }
    push(x) { this.in.push(x); }
    pop() { this.peak(); return this.out.pop(); }
    peek() { if (!this.out.length) while (this.in.length) this.out.push(this.in.pop()); return this.out[this.out.length-1]; }
    empty() { return !this.in.length && !this.out.length; }
}
```

复杂度：均摊时间 $O(1)$ ，空间 $O(n)$

74. 最小栈 ●

频度：99 | 难度：中等 | LeetCode 155

题意：支持 push/pop/top/getMin 都是 $O(1)$ 的栈。

💡 一句话思路：辅助栈同步存当前最小值。

🧠 记忆口诀：双栈同步，一个存值一个存最小

⚡ 速记模板：

```
class MinStack {
    constructor() { this.stack = []; this.minStack = [Infinity]; }
    push(x) { this.stack.push(x); this.minStack.push(Math.min(x, this.minStack[this.minStack.length-1])); }
    pop() { this.stack.pop(); this.minStack.pop(); }
    top() { return this.stack[this.stack.length-1]; }
    getMin() { return this.minStack[this.minStack.length-1]; }
}
```

复杂度：时间 $O(1)$ ，空间 $O(n)$

75. 滑动窗口最大值 ●

频度：151 | 难度：困难 | LeetCode 239

题意：滑动窗口中的最大值。

💡 一句话思路：单调递减双端队列，队头始终是最大值索引。

🧠 记忆口诀：队列递减存索引，新来弹小再入队，过期队头要弹出

⚡ 速记模板：

```
function maxSlidingWindow(nums, k) {
  const result = [], deque = [];
  for (let i = 0; i < nums.length; i++) {
    while (deque.length && nums[deque[deque.length-1]] < nums[i]) deque.pop();
    deque.push(i);
    if (deque[0] <= i - k) deque.shift();
    if (i >= k - 1) result.push(nums[deque[0]]);
  }
  return result;
}
```

复杂度：时间 $O(n)$ ，空间 $O(k)$

九、排序与 TopK

💡 **万能模板：**TopK 问题 = 快排 partition 或最小/大堆。只要目标是第 K 个，通常不需要完全排序。

💡 **记忆口诀：**TopK 用快选或堆，区间先排序

⚠️ **常见陷阱：**

- 快排 partition：pivot 选最后一个，i 指向小于 pivot 的区域末尾
- 合并区间：必须先按左端点排序

76. 手撕快速排序 🟡

题意：实现快速排序。

💡 **一句话思路：**选 pivot，partition 分成小于和大于两部分，递归排序。

💡 **记忆口诀：**选基准，分左右，递归排

⚡ **速记模板：**

```
function quickSort(arr, l = 0, r = arr.length - 1) {
  if (l >= r) return;
  const pivot = partition(arr, l, r);
  quickSort(arr, l, pivot - 1);
  quickSort(arr, pivot + 1, r);
}
function partition(arr, l, r) {
  const pivot = arr[r];
  let i = l;
  for (let j = l; j < r; j++) {
    if (arr[j] < pivot) { [arr[i], arr[j]] = [arr[j], arr[i]]; i++; }
  }
  [arr[i], arr[r]] = [arr[r], arr[i]];
  return i;
}
```

复杂度：时间 $O(n \log n)$ 平均，空间 $O(\log n)$

77. 数组中的第 K 个最大元素 ●

频度：595 | 难度：中等 | LeetCode 215

题意：未排序数组中第 K 个最大元素。

💡 一句话思路：快排 partition，找到 $\text{target} = \text{nums.length} - k$ 的位置。

🧠 记忆口诀：快选找第 K，partition 定位

⚡ 速记模板：

```
function findKthLargest(nums, k) {
  const target = nums.length - k;
  let l = 0, r = nums.length - 1;
  while (l < r) {
    const pivot = partition(nums, l, r);
    if (pivot === target) return nums[pivot];
    pivot < target ? l = pivot + 1 : r = pivot - 1;
  }
  return nums[l];
}

function partition(arr, l, r) {
  const pivot = arr[r]; let i = l;
  for (let j = l; j < r; j++) {
    if (arr[j] < pivot) { [arr[i], arr[j]] = [arr[j], arr[i]]; i++; }
  }
  [arr[i], arr[r]] = [arr[r], arr[i]]; return i;
}
```

复杂度：时间 $O(n)$ 平均，空间 $O(1)$

78. 合并 K 个排序链表 ●

频度：251 | 难度：困难 | LeetCode 23

题意：合并 K 个升序链表。

💡 一句话思路：分治法，两两合并，直到只剩一个。

🧠 记忆口诀：分治两两合，合到剩一个

⚡ 速记模板：

```
function mergeKLists(lists) {
  if (!lists.length) return null;
  while (lists.length > 1) {
    const merged = [];
    for (let i = 0; i < lists.length; i += 2) {
      merged.push(mergeTwoLists(lists[i], i+1 < lists.length ? lists[i+1] : null));
    }
    lists = merged;
  }
  return lists[0];
}
```

复杂度：时间 $O(n \log k)$ ，空间 $O(1)$

79. 合并区间 ●

频度：237 | 难度：中等 | LeetCode 56

题意：合并所有重叠区间。

💡 **一句话思路**：按左端点排序，遍历合并重叠区间。

💬 **记忆口诀**：排序后，重叠就合并右端点取大

⚡ **速记模板**：

```
function merge(intervals) {
  intervals.sort((a, b) => a[0] - b[0]);
  const result = [intervals[0]];
  for (let i = 1; i < intervals.length; i++) {
    const last = result[result.length - 1];
    if (intervals[i][0] <= last[1]) last[1] = Math.max(last[1], intervals[i][1]);
    else result.push(intervals[i]);
  }
  return result;
}
```

复杂度：时间 $O(n \log n)$ ，空间 $O(n)$

80. 最大数 ●

频度：82 | 难度：中等 | LeetCode 179

题意：数组拼成最大的数。

💡 **一句话思路**：自定义排序， $a+b > b+a$ 则 a 排在 b 前面。

💬 **记忆口诀**：拼接比较，大的放前面

⚡ **速记模板**：

```
function largestNumber(nums) {  
  const sorted = nums.sort((a, b) => (''+b+a).localeCompare(''+a+b));  
  const result = sorted.join('');  
  return result[0] === '0' ? '0' : result;  
}
```

复杂度：时间 $O(n \log n)$ ，空间 $O(n)$

十、回溯与递归

💡 **万能模板**：回溯 = 做选择 → 递归 → 撤销选择。关注路径、选择列表和结束条件。

🧠 **记忆口诀**：选、递、撤，三步走；剪枝去重不能少

⚠️ **常见陷阱**：

- 每次 push 后必须 pop，保证回到进入本层前的状态
- 有重复元素要先排序，同层跳过相同元素
- 组合问题传 start 避免重复选择

81. 全排列 🟡

频度：296 | 难度：中等 | LeetCode 46

题意：无重复数字的全排列。

💡 **一句话思路**：回溯 + used 数组标记已使用。

🧠 **记忆口诀**：枚举所有没用过的，用了标记，递归后撤销

⚡ **速记模板**：

```
function permute(nums) {  
  const result = [];  
  function backtrack(path, used) {  
    if (path.length === nums.length) { result.push([...path]); return; }  
    for (let i = 0; i < nums.length; i++) {  
      if (used[i]) continue;  
      path.push(nums[i]); used[i] = true;  
      backtrack(path, used);  
      path.pop(); used[i] = false;  
    }  
  }  
  backtrack([], []);  
  return result;  
}
```

复杂度：时间 $O(n \times n!)$ ，空间 $O(n)$

82. 组合总和 ●

频度： 94 | 难度： 中等 | LeetCode 39

题意：找出所有和为 target 的组合（数字可重复使用）。

💡 **一句话思路**：回溯，从 start 开始枚举，sum 超过 target 就剪枝。

🧠 **记忆口诀**：从 start 枚举，可重复用，超了就剪

⚡ **速记模板**：

```
function combinationSum(candidates, target) {
  const result = [];
  function backtrack(start, path, sum) {
    if (sum === target) { result.push([...path]); return; }
    if (sum > target) return;
    for (let i = start; i < candidates.length; i++) {
      path.push(candidates[i]);
      backtrack(i, path, sum + candidates[i]);
      path.pop();
    }
  }
  backtrack(0, [], 0);
  return result;
}
```

复杂度：时间 $O(2^n)$ ，空间 $O(\text{target})$

83. 子集 ●

频度： 104 | 难度： 中等 | LeetCode 78

题意：返回数组的所有子集。

💡 **一句话思路**：回溯，每个元素选或不选，每次进入都记录当前路径。

🧠 **记忆口诀**：每层都记录，从 start 枚举选或不选

⚡ **速记模板**：

```
function subsets(nums) {
  const result = [];
  function backtrack(start, path) {
    result.push([...path]);
    for (let i = start; i < nums.length; i++) {
      path.push(nums[i]);
      backtrack(i + 1, path);
      path.pop();
    }
  }
  backtrack(0, []);
  return result;
}
```

复杂度：时间 $O(n \times 2^n)$ ，空间 $O(n)$

84. 括号生成 ● (见 63 题)

85. 单词拆分 ●

频度：69 | 难度：中等 | LeetCode 139

题意：字符串能否被字典中的单词拆分。

💡 一句话思路：DP， $dp[i]$ 表示前 i 个字符能否拆分。

🗯️ 记忆口诀： $dp[i]$ = 存在 j 使 $dp[j]$ 为真且 $s[j:i]$ 在字典中

⚡ 速记模板：

```
function wordBreak(s, wordDict) {
  const set = new Set(wordDict);
  const dp = new Array(s.length + 1).fill(false);
  dp[0] = true;
  for (let i = 1; i <= s.length; i++) {
    for (let j = 0; j < i; j++) {
      if (dp[j] && set.has(s.substring(j, i))) { dp[i] = true; break; }
    }
  }
  return dp[s.length];
}
```

复杂度：时间 $O(n^2)$ ，空间 $O(n)$

十一、DFS/BFS/图

💡 万能模板：网格 DFS = 遍历四个方向 + 标记已访问。BFS = 队列层序扩散。拓扑排序 = 入度 + 队列。

🗯️ 记忆口诀：DFS 走到底，BFS 一层层，拓扑看入度

⚠️ 常见陷阱：

- 访问过的节点要立即标记，避免重复入队
- 网格题先判断越界和障碍
- 拓扑排序入度为 0 的节点入队

86. 岛屿数量 ●

频度：324 | 难度：中等 | LeetCode 200

题意：网格中 1 表示陆地，0 表示水，求岛屿数量。

💡 **一句话思路**：遍历网格，遇到 1 就计数 +1 并用 DFS 把整个岛屿淹没（设为 0）。

🧠 **记忆口诀**：遇 1 计数加一，DFS 淹没整片岛

⚡ **速记模板**：

```
function numIslands(grid) {
  let count = 0;
  const m = grid.length, n = grid[0].length;
  function dfs(i, j) {
    if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] === '0') return;
    grid[i][j] = '0';
    dfs(i-1,j); dfs(i+1,j); dfs(i,j-1); dfs(i,j+1);
  }
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++)
      if (grid[i][j] === '1') { count++; dfs(i, j); }
  return count;
}
```

复杂度：时间 $O(mn)$ ，空间 $O(mn)$

87. 岛屿的最大面积 ●

频度：90 | 难度：中等 | LeetCode 695

题意：求岛屿的最大面积。

💡 **一句话思路**：和岛屿数量类似，DFS 返回面积。

🧠 **记忆口诀**：DFS 返回 1 + 四周面积

⚡ **速记模板**：

```
function maxAreaOfIsland(grid) {
  const m = grid.length, n = grid[0].length;
  let maxArea = 0;
  function dfs(i, j) {
    if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] === 0) return 0;
    grid[i][j] = 0;
    return 1 + dfs(i-1,j) + dfs(i+1,j) + dfs(i,j-1) + dfs(i,j+1);
  }
  for (let i = 0; i < m; i++)
    for (let j = 0; j < n; j++)
      if (grid[i][j] === 1) maxArea = Math.max(maxArea, dfs(i, j));
  return maxArea;
}
```

复杂度：时间 $O(mn)$ ，空间 $O(mn)$

88. 课程表

频度：65 | 难度：中等 | LeetCode 207

题意：判断能否完成所有课程（有向图是否有环）。

 **一句话思路**：拓扑排序，BFS 计算入度，能访问所有节点就无环。

 **记忆口诀**：算入度，零入度入队，出队减邻居入度

 **速记模板**：

```
function canFinish(numCourses, prerequisites) {
  const inDegree = new Array(numCourses).fill(0);
  const graph = Array.from({length: numCourses}, () => []);
  for (const [course, pre] of prerequisites) { graph[pre].push(course); inDegree[course]++; }
  const queue = [];
  for (let i = 0; i < numCourses; i++) if (inDegree[i] === 0) queue.push(i);
  let visited = 0;
  while (queue.length) {
    const node = queue.shift(); visited++;
    for (const next of graph[node]) { inDegree[next]--; if (inDegree[next] === 0) queue.push(next); }
  }
  return visited === numCourses;
}
```

复杂度：时间 $O(V+E)$ ，空间 $O(V+E)$

89. 二叉树的右视图 ● (见 27 题)

90. 二叉树的层序遍历 ● (见 24 题)

十二、设计题

💡 万能模板：设计题 = 组合多个数据结构，让每个 API 复杂度稳定。

🧠 记忆口诀：哈希管定位，链表管顺序，双栈模拟队列

⚠️ 常见陷阱：

- LRU：Map 的 delete + set 可以模拟移到末尾
- Rand10：拒绝采样，超出范围就重试

91. LRU 缓存机制 ●

频度：917 | 难度：中等 | LeetCode 146

题意：实现 get 和 put 都是 O(1) 的 LRU 缓存。

💡 一句话思路：Map 天然有序，delete + set 可以把键移到末尾，末尾是最久未使用。

🧠 记忆口诀：Map 存数据，get 时删了重设移到最后，超容量删第一个

⚡ 速记模板：

```
class LRUCache {
  constructor(capacity) { this.capacity = capacity; this.cache = new Map(); }
  get(key) {
    if (!this.cache.has(key)) return -1;
    const val = this.cache.get(key);
    this.cache.delete(key); this.cache.set(key, val);
    return val;
  }
  put(key, value) {
    if (this.cache.has(key)) this.cache.delete(key);
    this.cache.set(key, value);
    if (this.cache.size > this.capacity) this.cache.delete(this.cache.keys().next().value);
  }
}
```

复杂度：时间 O(1)，空间 O(capacity)

92. 用栈实现队列 ● (见 73 题)

93. 用 Rand7 实现 Rand10 ●

频度: 94 | 难度: 中等 | LeetCode 470

题意: 只能调用 rand7, 生成 1-10 均匀随机数。

💡 **一句话思路:** 拒绝采样, $(\text{rand7}() - 1) * 7 + \text{rand7}()$ 生成 1-49, 取 1-40 映射到 1-10。

💬 **记忆口诀:** 7 进位生成 1-49, 取前 40 模 10

⚡ **速记模板:**

```
function rand10() {  
  while (true) {  
    const num = (rand7() - 1) * 7 + rand7();  
    if (num <= 40) return ((num - 1) % 10) + 1;  
  }  
}
```

复杂度: 时间 $O(1)$ 期望, 空间 $O(1)$

十三、其他经典

💡 **万能模板:** 这类题更看重约束洞察: 能否单趟维护答案、能否原地交换、能否把多步操作拆成可验证的不变量。

💬 **记忆口诀:** 看约束, 找规律, 单趟或原地

⚠️ **常见陷阱:**

- 原地矩阵操作: 先转置再翻转, 或分层旋转
- 投票算法: count 为 0 时换候选人

94. 买卖股票的最佳时机 ●

频度: 278 | 难度: 简单 | LeetCode 121

题意: 只能买卖一次, 最大利润。

💡 **一句话思路:** 遍历维护最小价格和最大利润。

💬 **记忆口诀:** 记录最小价, 每天算利润取大

⚡ **速记模板:**

```
function maxProfit(prices) {
  let minPrice = Infinity, maxProfit = 0;
  for (const price of prices) {
    if (price < minPrice) minPrice = price;
    else maxProfit = Math.max(maxProfit, price - minPrice);
  }
  return maxProfit;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

95. 买卖股票的最佳时机 II ●

频度：88 | 难度：中等 | LeetCode 122

题意：可以多次买卖，最大利润。

💡 **一句话思路**：贪心，只要今天比昨天高就卖。

🧠 **记忆口诀**：涨了就拿，累加所有正差

⚡ **速记模板**：

```
function maxProfit(prices) {
  let profit = 0;
  for (let i = 1; i < prices.length; i++)
    if (prices[i] > prices[i-1]) profit += prices[i] - prices[i-1];
  return profit;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

96. 多数元素 ●

频度：67 | 难度：简单 | LeetCode 169

题意：找出现次数超过 $n/2$ 的元素。

💡 **一句话思路**：Boyer-Moore 投票算法，count 为 0 换候选人。

🧠 **记忆口诀**：同则加，异则减，为零换人

⚡ **速记模板**：

```
function majorityElement(nums) {
  let count = 0, candidate = null;
  for (const num of nums) {
    if (count === 0) candidate = num;
    count += (num === candidate) ? 1 : -1;
  }
  return candidate;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

97. 缺失的第一个正数 ●

频度：113 | 难度：困难 | LeetCode 41

题意：未排序数组中缺失的最小正整数。

💡 **一句话思路**：原地哈希，把数字 i 放到索引 $i-1$ 的位置。

💬 **记忆口诀**：数字归位， i 放 $i-1$ ，第一个不对的就是答案

⚡ **速记模板**：

```
function firstMissingPositive(nums) {
  const n = nums.length;
  for (let i = 0; i < n; i++) {
    while (nums[i] > 0 && nums[i] <= n && nums[nums[i]-1] !== nums[i]) {
      [nums[nums[i]-1], nums[i]] = [nums[i], nums[nums[i]-1]];
    }
  }
  for (let i = 0; i < n; i++) if (nums[i] !== i + 1) return i + 1;
  return n + 1;
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

98. 旋转图像 ●

频度：83 | 难度：中等 | LeetCode 48

题意： $n \times n$ 矩阵顺时针旋转 90 度（原地）。

💡 **一句话思路**：先转置（行列交换），再每行反转。

💬 **记忆口诀**：转置加反转

⚡ **速记模板**：

```
function rotate(matrix) {
  const n = matrix.length;
  for (let i = 0; i < n; i++)
    for (let j = i + 1; j < n; j++)
      [matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]];
  for (let i = 0; i < n; i++) matrix[i].reverse();
}
```

复杂度：时间 $O(n^2)$ ，空间 $O(1)$

99. 螺旋矩阵 ●

频度：252 | 难度：中等 | LeetCode 54

题意：顺时针螺旋顺序返回矩阵元素。

💡 一句话思路：四个边界 top/bottom/left/right，按右-下-左-上顺序遍历。

💡 记忆口诀：四边界，右左上下，每走完一条边收缩

⚡ 速记模板：

```
function spiralOrder(matrix) {
  if (!matrix.length) return [];
  const result = [];
  let top = 0, bottom = matrix.length - 1, left = 0, right = matrix[0].length - 1;
  while (top <= bottom && left <= right) {
    for (let i = left; i <= right; i++) result.push(matrix[top][i]); top++;
    for (let i = top; i <= bottom; i++) result.push(matrix[i][right]); right--;
    if (top <= bottom) { for (let i = right; i >= left; i--) result.push(matrix[bottom][i]);
    bottom--; }
    if (left <= right) { for (let i = bottom; i >= top; i--) result.push(matrix[i][left]);
    left++; }
  }
  return result;
}
```

复杂度：时间 $O(mn)$ ，空间 $O(1)$

100. 下一个排列 ●

频度：136 | 难度：中等 | LeetCode 31

题意：字典序中下一个更大的排列。

💡 一句话思路：从右找第一个下降点 i ，从右找第一个大于 $nums[i]$ 的 j 交换，然后反转 i 后面。

💡 记忆口诀：右找下降，右找更大交换，后面反转

⚡ 速记模板：

```
function nextPermutation(nums) {
  let i = nums.length - 2;
  while (i >= 0 && nums[i] >= nums[i+1]) i--;
  if (i >= 0) {
    let j = nums.length - 1;
    while (nums[j] <= nums[i]) j--;
    [nums[i], nums[j]] = [nums[j], nums[i]];
  }
  let l = i + 1, r = nums.length - 1;
  while (l < r) { [nums[l], nums[r]] = [nums[r], nums[l]]; l++; r--; }
}
```

复杂度：时间 $O(n)$ ，空间 $O(1)$

恭喜完成 CodeTop 100 题解学习!

复习建议

- 1. **每周复盘**：挑 10 道错题重写，重点看边界条件
- 2. **模拟面试**：限时 45 分钟完成 1 道中等题，训练手写速度
- 3. **模板迁移**：把哈希、双指针、DP、DFS/BFS、回溯的模板练到肌肉记忆
- 4. **举一反三**：同题型换皮（如"两数之和"→"三数之和"）要能快速识别

面试技巧

- **先说思路**：写代码前先和面试官确认思路，避免写偏
- **边写边讲**：解释关键步骤和复杂度，展现思考过程
- **主动补边界**：空数组、单元素、重复值、负数、溢出都要考虑到
- **写不完也没关系**：把核心逻辑写清楚，边界可以口头补充

祝你面试顺利，拿到心仪 Offer! 🚀

导航

 [上一章：前端工程化](#)

 [返回主指南](#)

 [下一章：计算机网络](#)